

PRESENTED BY:



CODEBLOODED

Gamified Financial Visualizer

EPI·USE[®]



WHAT IS GAMIFIED FINANCIAL VISUALIZER?

Content

GFV is a gamified financial management system that turns your finances into an interactive city and board game experience, helping users learn, plan, and grow their wealth while having fun.



Uniqueness

- Gamification with a 3D city and Monopoly-style financial game
- AI-driven personalized insights
- Engaging and educational for users of all ages



Use Cases

A decorative graphic featuring pixelated clouds in various shades of blue and white. The clouds are arranged in a way that they appear to be floating around a central dark blue rounded rectangle. There are several cloud clusters: one large one on the right, one on the left, and a row of smaller ones at the bottom. The overall style is retro and digital.

WHAT CAN IT DO?

Create a budget

Transactions

- Create a budget for a transaction category to track expenses.

Create a transaction

Transactions

- Manually input and categorize transactions for tracking.

Set up a goal

Goal Management

As a user, I want to create savings goals so I can track toward big purchases.

Profile Customization

Profile

- Change my profile avatar (to a preset) and username.

Community Creation

Community

- Create a community, invite friends, and tackle challenges together.

Learning

Goal Management

- Complete financial literacy modules and quizzes to earn points.

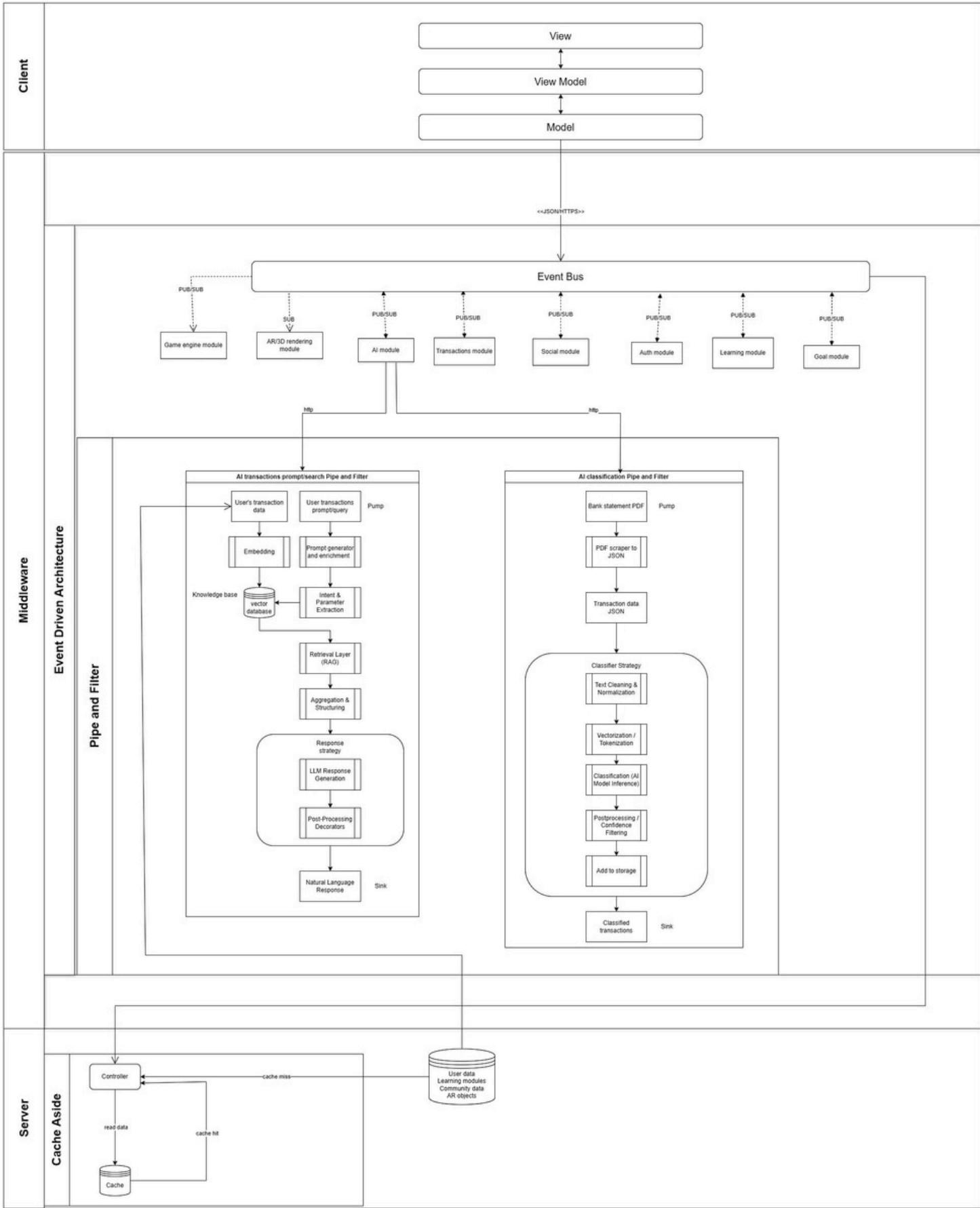


ARCHITECTURE

A pixelated, retro-style illustration of a landscape with blue and white clouds. The clouds are rendered in a blocky, pixelated manner, with some appearing as soft, billowing shapes and others as more defined, angular forms. They are scattered across the scene, with a larger cluster on the right and several smaller ones on the left and bottom. The colors are a mix of light blue, medium blue, and white, giving it a clean, digital feel.

Architecture

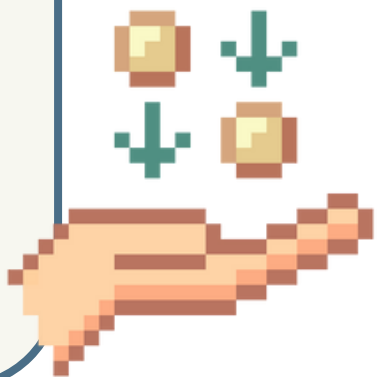
ARCHITECTURAL DIAGRAM (THREE-TIER)



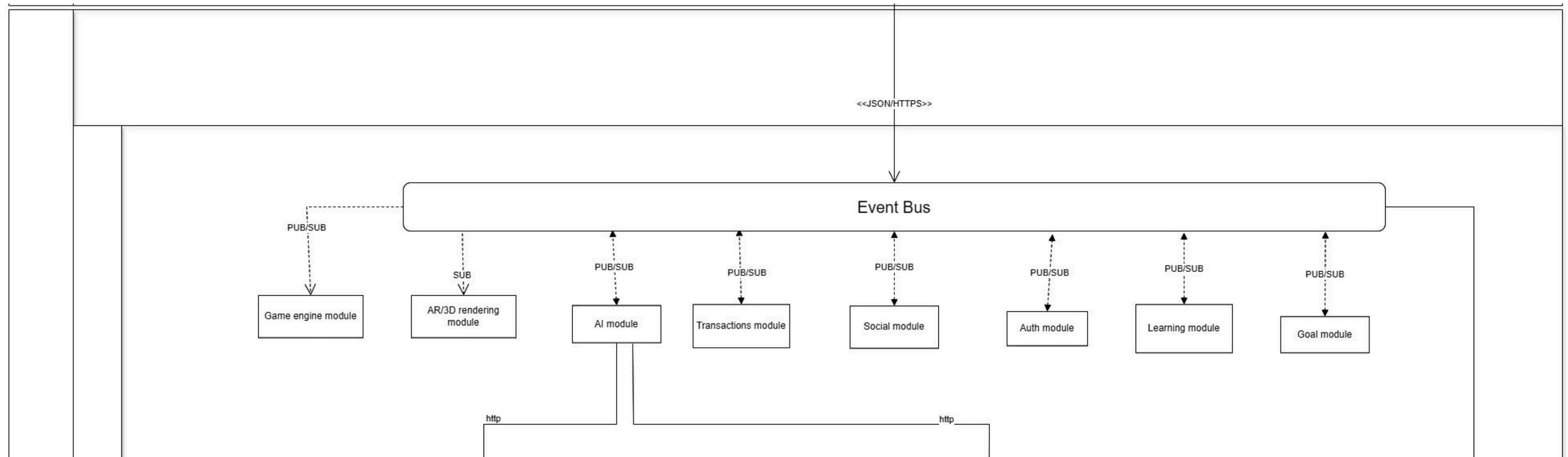
Three-tier Architecture

Our system uses a modular, client-middleware-server architecture. This provides:

- Security through controlled boundaries
- Scalability via async task processing
- Reliability with isolated failures
- Compatibility via independent module updates



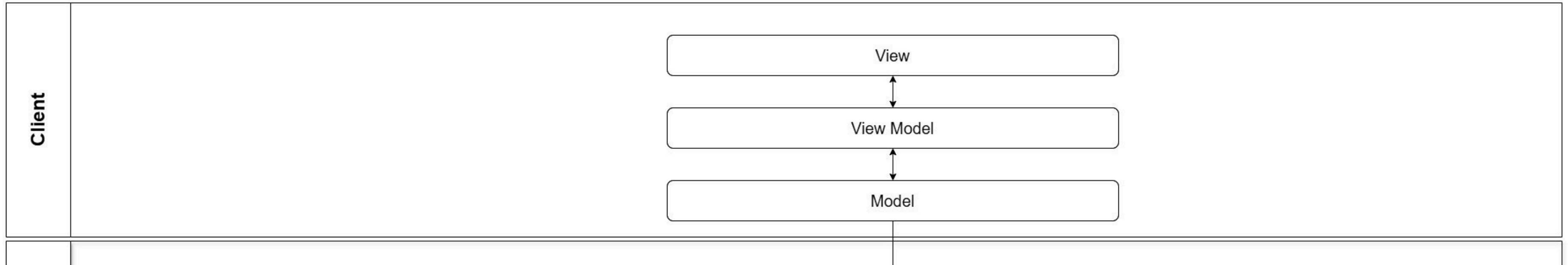
ARCHITECTURAL DIAGRAM (EVENT-DRIVEN)



Event-Driven Architecture

- Scalability: Loosely-coupled services
 - Reliability: Non-blocking, fault-tolerant queues
 - Usability: Instant user feedback
 - Maintainability: Pluggable logic without core changes
- The system uses an event-driven model. Real-time events (like a completed quiz) are broadcast to independent subscribers (e.g., notifications, AI).

ARCHITECTURAL DIAGRAM (MVVM)



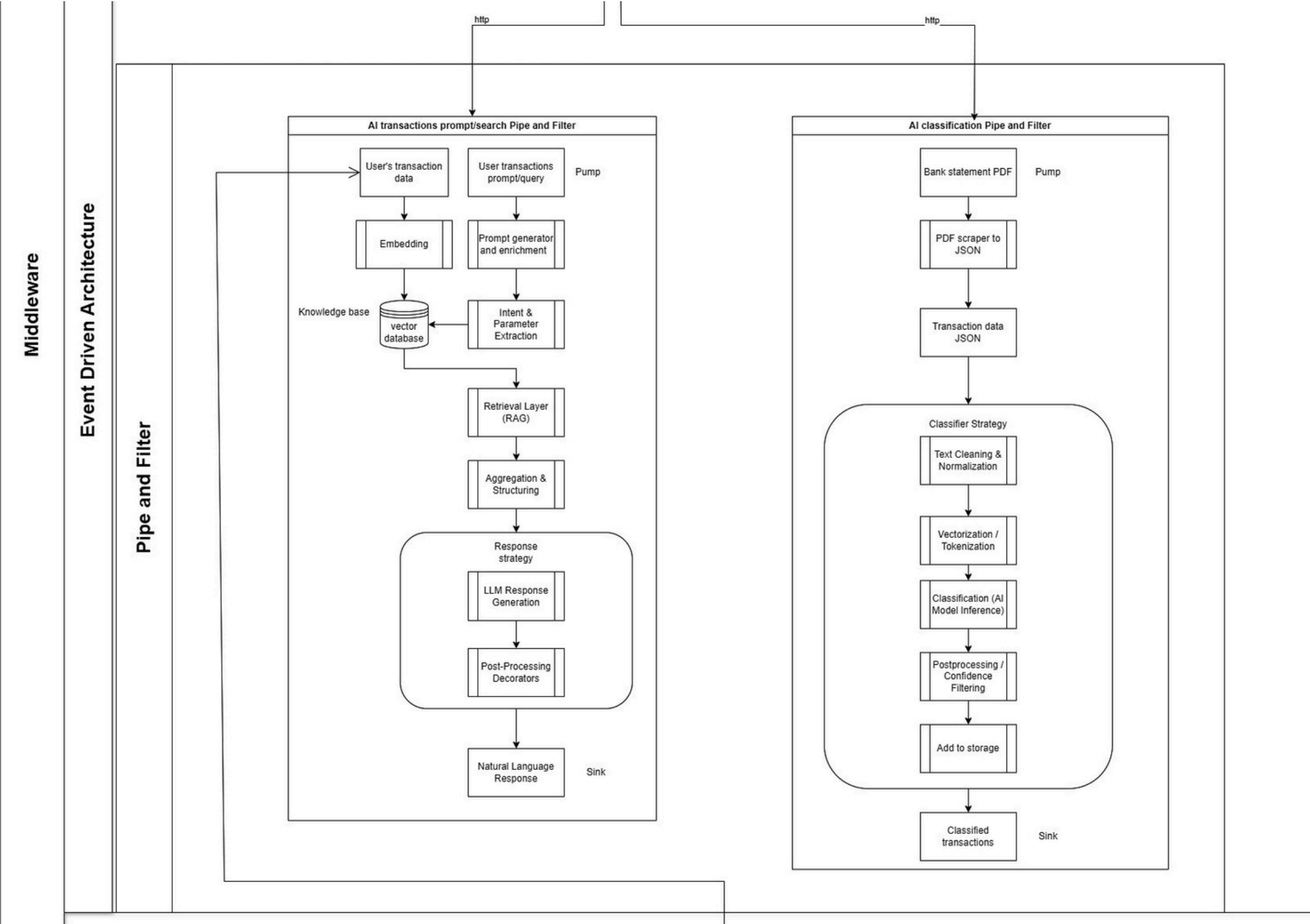
Model–View–ViewModel

A layered UI architecture separates input, state, and presentation logic.

Benefits:

- Performance: Targeted component re-renders
- Reliability: Type-safe, encapsulated logic
- Maintainability: Clear separation for easier testing
- Scalability: Composable features
- Accessibility: Built-in semantic HTML

ARCHITECTURAL DIAGRAM (PIPE-AND-FILTER)



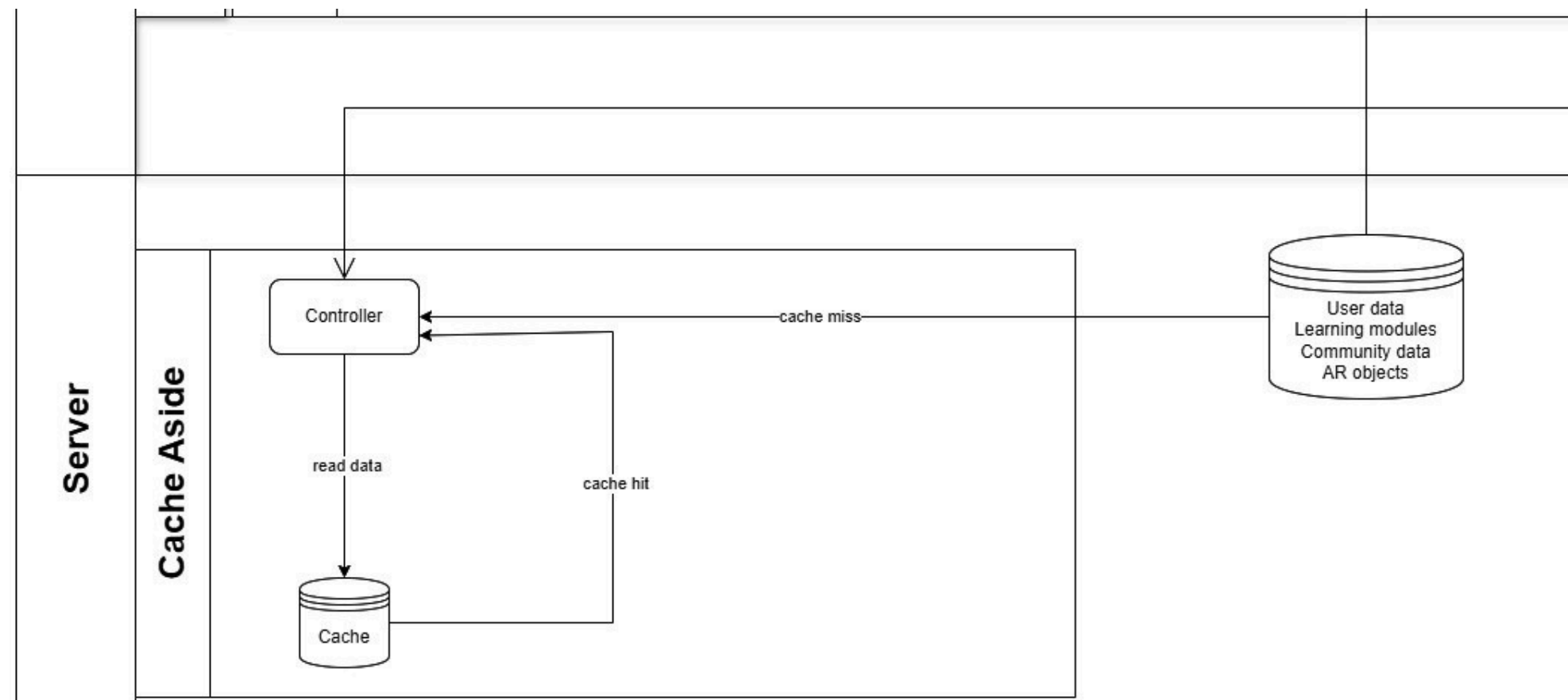
Pipe and Filter

A sequential filter pipeline processes data for AI and AR. Each filter handles a specific task like validation or transformation.

Benefits:

- Performance: Per-stage optimization
- Reliability: Validation and fallback
- Maintainability: Easy to test and extend
- Compliance: Legal filters (e.g., disclaimers)

ARCHITECTURAL DIAGRAM (CACHE ASIDE)



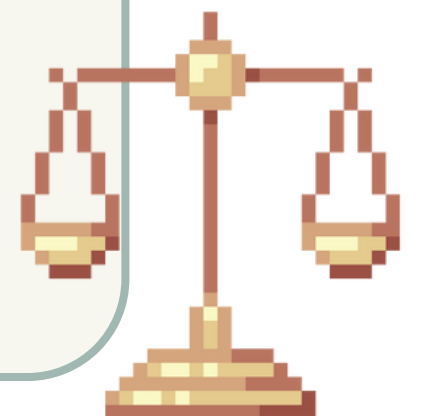
Cache Aside

The system uses a cache (e.g., Redis) for static data like goals.

Workflow: Check cache → return if found → otherwise, query DB → cache result → return.

Benefits:

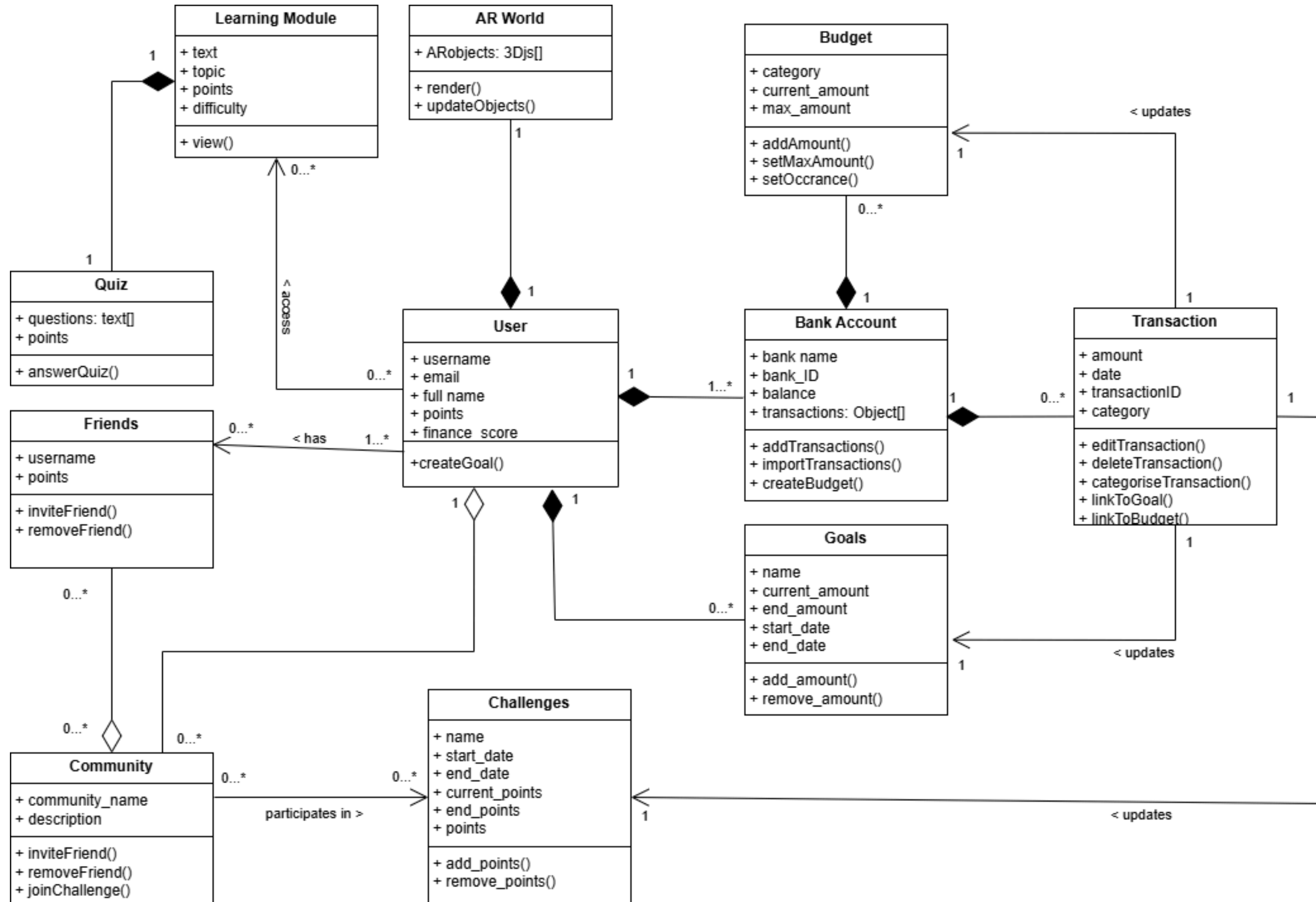
- Performance: Faster access (<50ms)
- Scalability: Reduces database load
- Reliability: Graceful degradation
- Maintainability: Transparent to services





Domain Model

DOMAIN MODEL V4



Project Management

A decorative graphic featuring pixelated clouds in shades of blue and white. The clouds are scattered around a central dark blue rounded rectangle, with some appearing behind it and others in front. The overall style is retro and digital.

SRCUM FRAMEWORK

AGILE SCRUM

lightweight, iterative, team-driven framework for delivering working software incrementally.

Sprint Structure

Sprint Duration: Weekly

- Sprint Planning
- Daily Standups
- Sprint Review
- Sprint Retrospective

Tools & Practices

- **Projecting Tracking:** GitHub Projects/ Issues/ Milestones and Notion
- **Version Control:** Git (main, dev and feature branches)
- **Documentation:** README, ERDs, SRS and API specs
- **Continuous Integration:** GitHub Actions for testing workflows

Notion Project Board



GROUP ORGANISATION

No Status4Estimate: 0

Gamified-Financial-Visualizer #430

Testing policy document

Gamified-Financial-Visualizer #437

Full system debug with deployment

Gamified-Financial-Visualizer #472

Complete tutorials page

Gamified-Financial-Visualizer #481

Check all CRUD operations integration

+ Add item

Todo2 / 5Estimate: 0

This item hasn't been started

Gamified-Financial-Visualizer #94

Revise event bus functions for transactions/goals and budget that IDs are sent through

Gamified-Financial-Visualizer #307

Check that community invites show up on the other end of communication

+ Add item

In Progress1 / 5Estimate: 0

This is actively being worked on

Gamified-Financial-Visualizer #474

Fix transactions page

+ Add item

In Progress1 / 5Estimate: 0

This is actively being worked on

Gamified-Financial-Visualizer #474

Fix transactions page

+ Add item

On Hold1Estimate: 0

This is on hold for the current iteration.

Gamified-Financial-Visualizer #49

Implement SQL query to get upcoming recurring transactions

+ Add item

Done383Estimate: 0

This has been completed

Gamified-Financial-Visualizer #229

Complete help page in-product feature

Gamified-Financial-Visualizer #248

Review weekly goal progress chart

Gamified-Financial-Visualizer #181

Integrate Settings Page

Gamified-Financial-Visualizer #244

Complete Settings Page Hotfixes

+ Add item

SOFTWARE DEVELOPMENT METHODOLOGY

AGILE

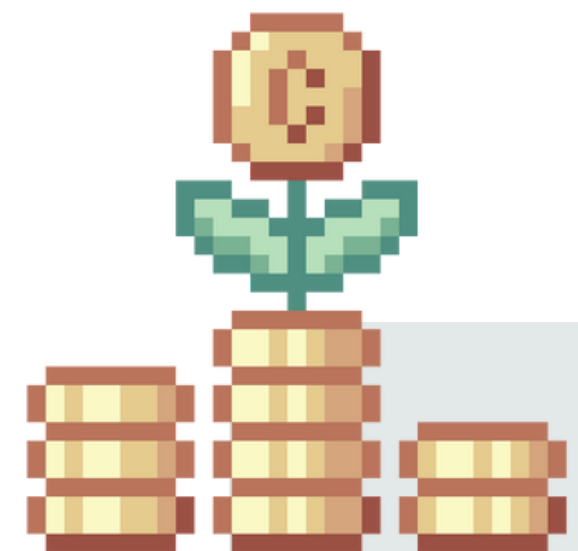
emphasizes collaboration, adaptability, and incremental delivery of valuable software

Key Principles

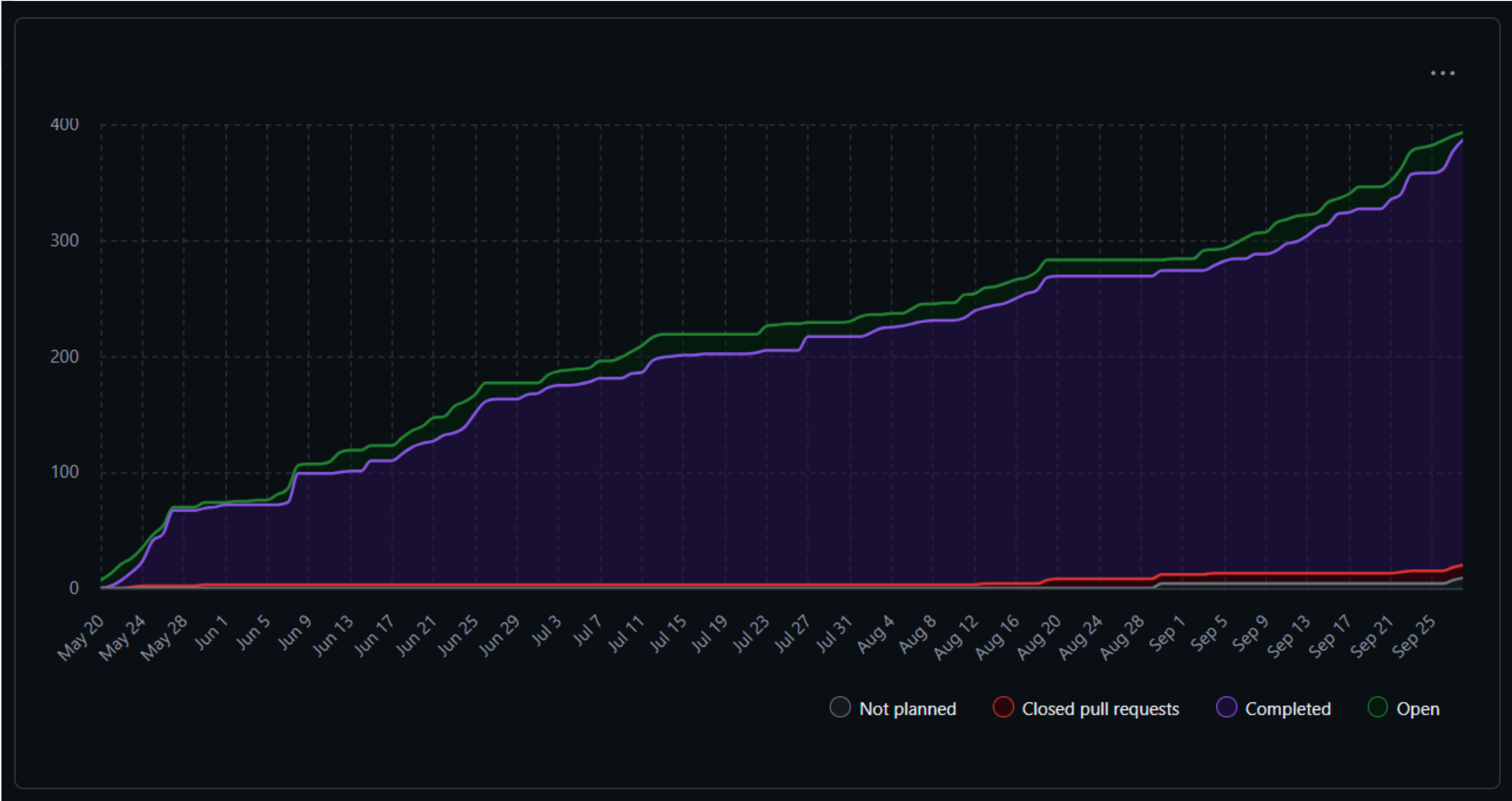
- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** by following a plan

Iterative Lifecycle

1. Requirement Gathering
2. Design & Planning
3. Implementation
4. Testing
5. Review & Feedback
6. Deployment



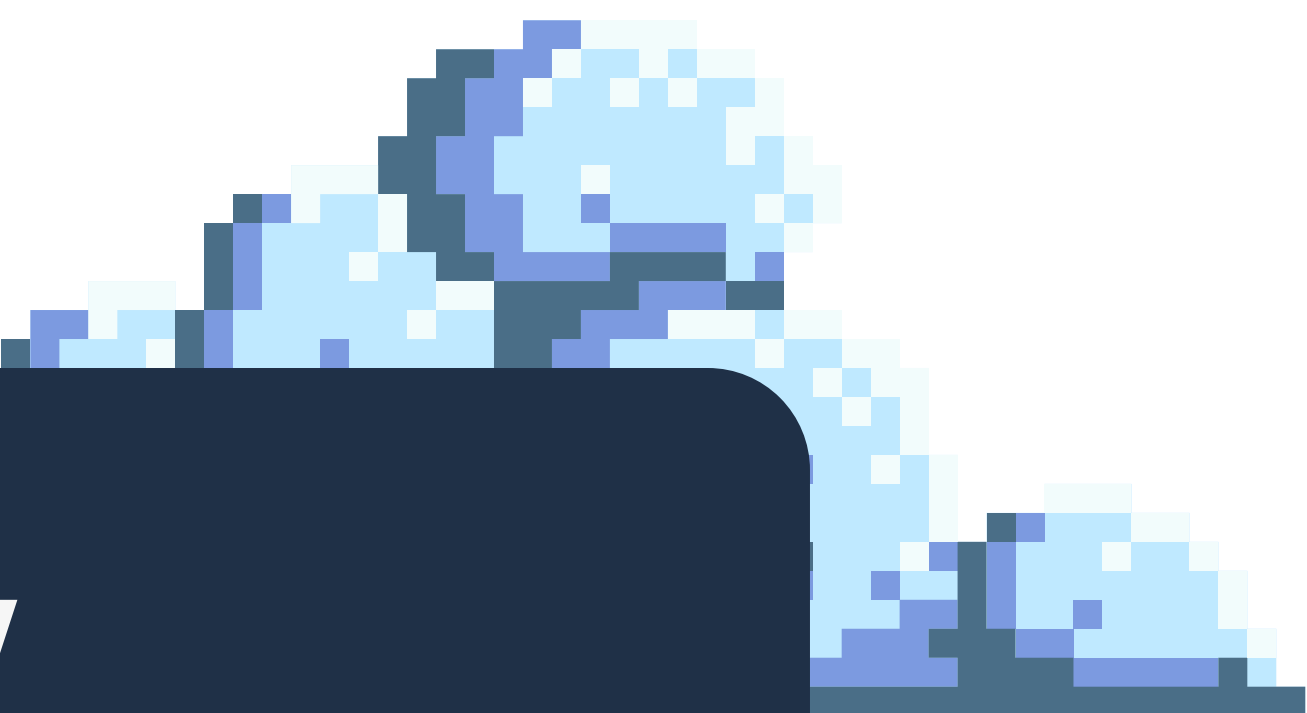
BURN UP CHART



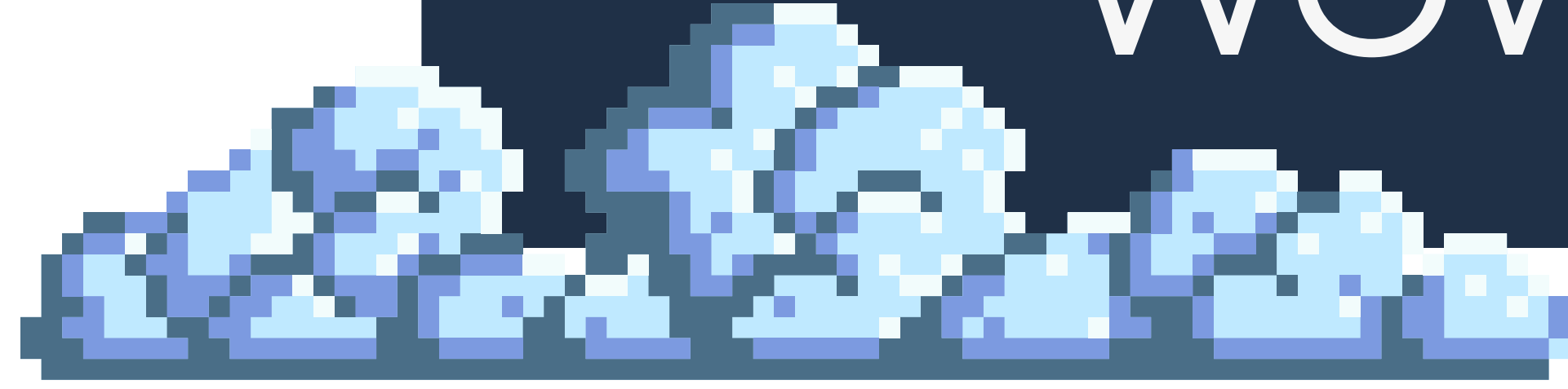
LIVE DEMO

Live Demo

WOW FACTORS



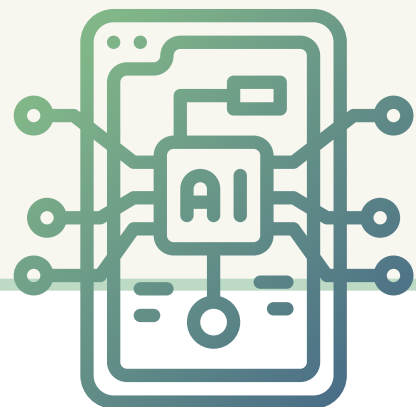
WOW



WOW FACTORS

ML: Insights

- Analyzes transactions and goals using ML
- Generates personalized insights and predictions
- Provides data-driven feedback to improve financial habits



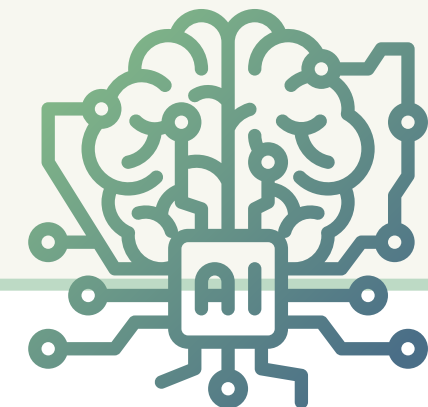
AI Chatbot

- Answers money questions in plain English
- Gives personalized budgeting & saving advice
- Translates complex data into simple insights



Game

- Interactive 3D cityscape for accounts and goals
- Monopoly-style board for financial actions
- Gamified experience to boost engagement



TESTING



Testing

UNIT TESTS

```
PASS api-backend/tests/services/community.test.ts (11.429 s)
PASS api-backend/tests/integration/auth.test.ts (12.723 s)
PASS api-backend/tests/services/auth.test.ts
PASS api-backend/tests/integration/community.test.ts (14.356 s)
PASS api-backend/tests/integration/insights.test.ts
PASS api-backend/tests/services/learning.test.ts (13.045 s)
PASS api-backend/tests/integration/transactions.test.ts (14.725 s)
PASS api-backend/tests/integration/goals.test.ts
PASS api-backend/tests/services/transactions.test.ts
PASS api-backend/tests/integration/achievement.test.ts
PASS api-backend/tests/services/insights.test.ts
PASS api-backend/tests/services/achievement.test.ts
PASS api-backend/tests/services/goals.test.ts (12.659 s)
PASS api-backend/tests/integration/learning.test.ts
```

```
Test Suites: 14 passed, 14 total
```

```
Tests:      431 passed, 431 total
```

```
Snapshots:  0 total
```

```
Time:       30.844 s
```

```
Ran all test suites.
```

INTEGRATION TESTS

```
► Run npm run test:int
```

```
> gfv-dev@1.0.0 test:int
```

```
> jest integration --setupFilesAfterEnv=./api-backend/tests/jest-setup.ts
```

```
PASS api-backend/tests/integration/auth.test.ts (11.589 s)
```

```
PASS api-backend/tests/integration/insights.test.ts
```

```
PASS api-backend/tests/integration/community.test.ts (13.412 s)
```

```
PASS api-backend/tests/integration/achievement.test.ts
```

```
PASS api-backend/tests/integration/transactions.test.ts (23.108 s)
```

```
PASS api-backend/tests/integration/goals.test.ts (12.145 s)
```

```
PASS api-backend/tests/integration/learning.test.ts (10.939 s)
```

```
Test Suites: 7 passed, 7 total
```

```
Tests:      241 passed, 241 total
```

```
Snapshots:  0 total
```

```
Time:       26.895 s
```

```
Ran all test suites matching /integration/i.
```

NON-FUNCTIONAL TESTING

Performance

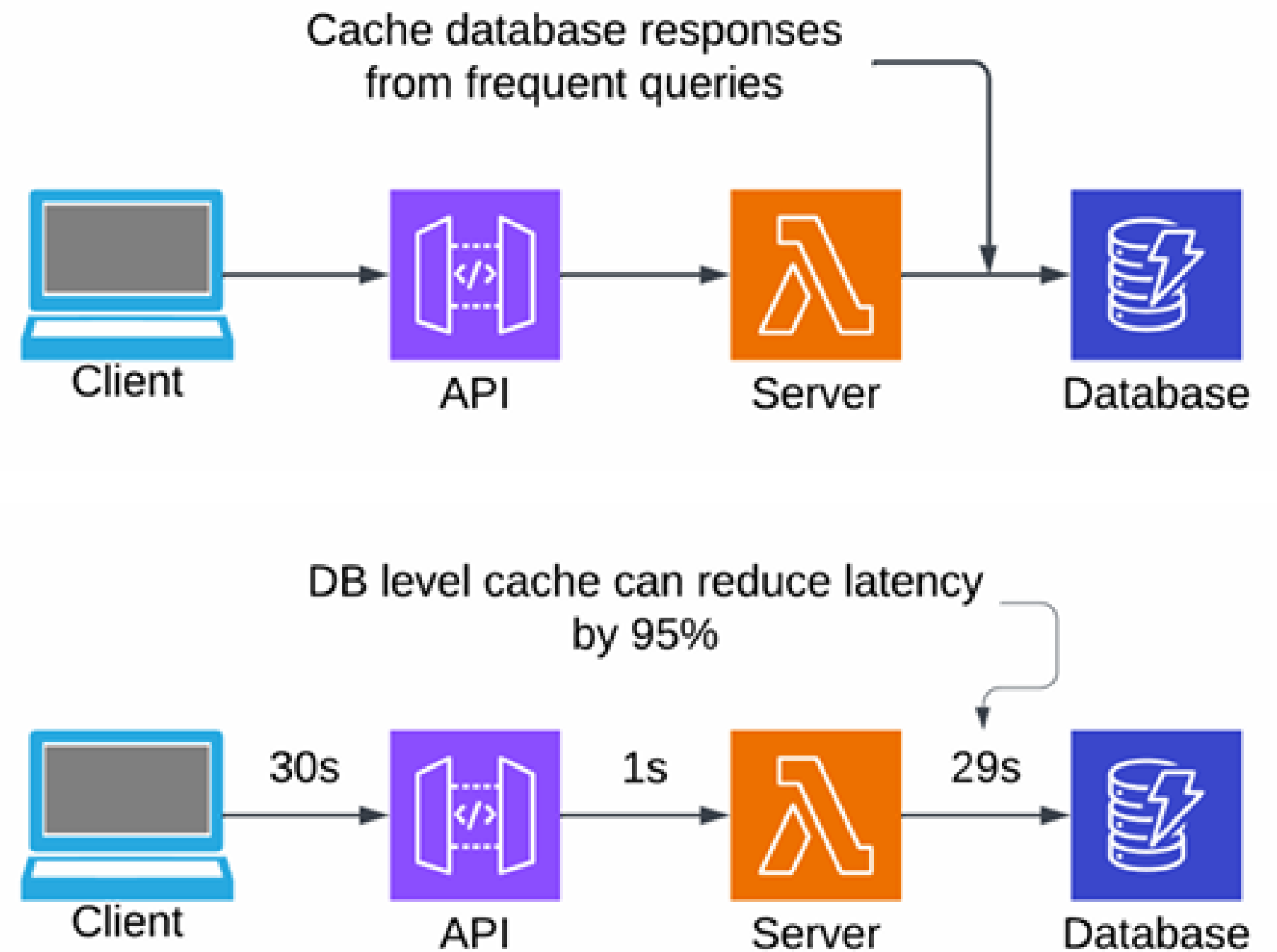
Objective: Ensure platform speed and responsiveness.

Method: **Lighthouse** audits on staging, testing key user flows under slow network/CPU conditions.

Results:

- Warm loads fast (<2s LCP, <50ms cached)
- Redis caching boosted dashboard speed
- Passed WCAG accessibility checks

Optimizations: Lazy loading, tree-shaking, asset preloading, ARIA improvements.



credit: @ 2025 Arjay McCandless

NON-FUNCTIONAL TESTING



METRICS Expand view

First Contentful Paint
1.4 s

Largest Contentful Paint
12.9 s

about:blank 1/5

9/25/25, 9:49 PM about:blank

Total Blocking Time
5,720 ms

Speed Index
6.3 s

Cumulative Layout Shift
0



Later this year, insights will replace performance audits. [Learn more and provide feedback here.](#) [Go back to audits](#)



METRICS Expand view

First Contentful Paint
1.4 s

Largest Contentful Paint
7.4 s

about:blank 1/4

9/25/25, 9:54 PM about:blank

Total Blocking Time
6,570 ms

Speed Index
9.4 s

Cumulative Layout Shift
0



about:blank 1/5

9/25/25, 9:59 PM about:blank

METRICS Expand view

First Contentful Paint
1.0 s

Largest Contentful Paint
19.8 s

Total Blocking Time
5,580 ms

Speed Index
6.4 s

Cumulative Layout Shift
0.002



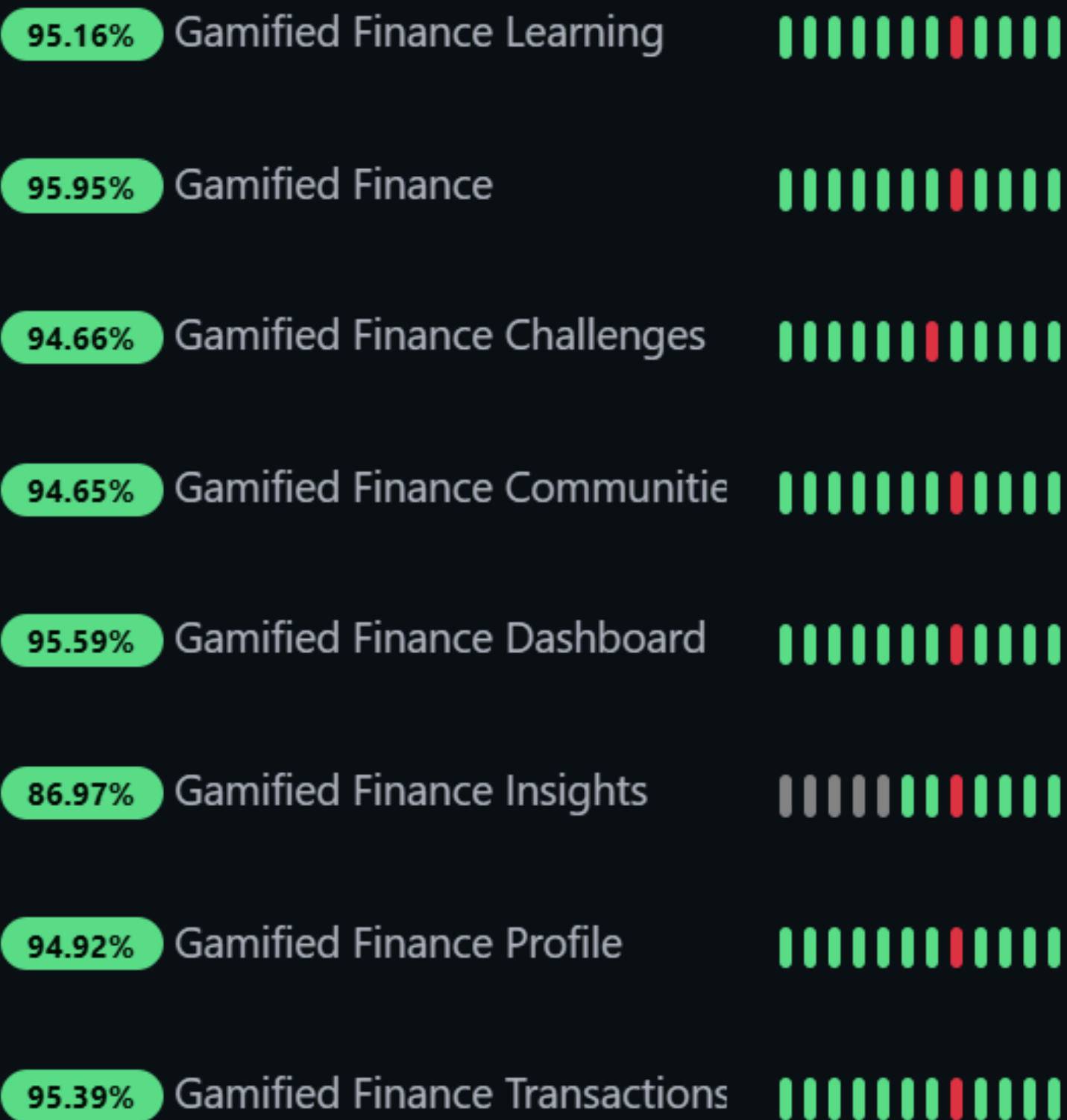
NON-FUNCTIONAL TESTING

Availability

Objective: Ensure 95% service availability.
Method: Comprehensive monitoring of API, DB, Redis, and sockets using **Uptime Kuma**.

- Results:
- 90% uptime across all services
 - Response times <500ms average
 - Reliable socket auto-reconnection

Optimizations: DB redundancy, heartbeat monitoring, service isolation.



NON-FUNCTIONAL TESTING

Usability

Objective: Validate intuitive and engaging UI.

Method: Remote usability testing with 8 participants (ages 18-24).

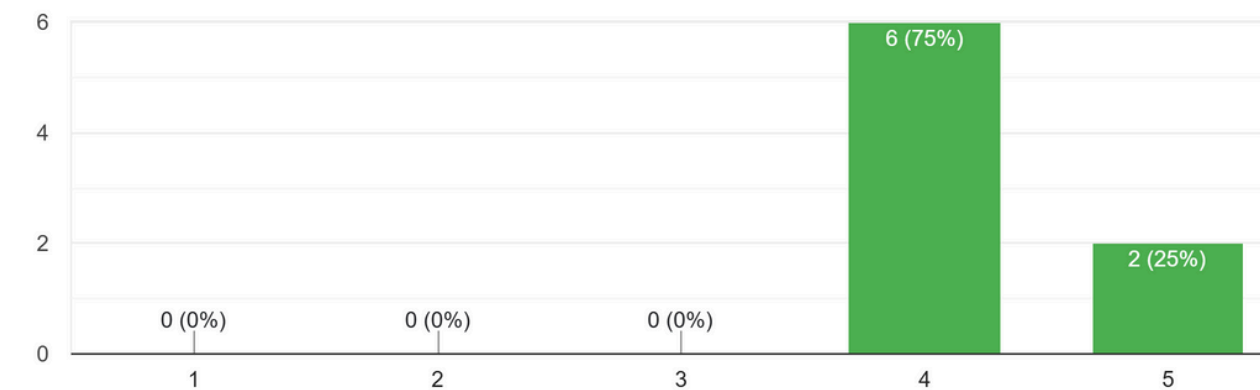
Results:

- 100% positive navigation rating (4-5/5)
- High engagement (75% engaging, 25% very engaging)
- 100% overall satisfaction (4-5/5)

Solutions: Improved dark mode contrast, added achievement modals, and loading states.

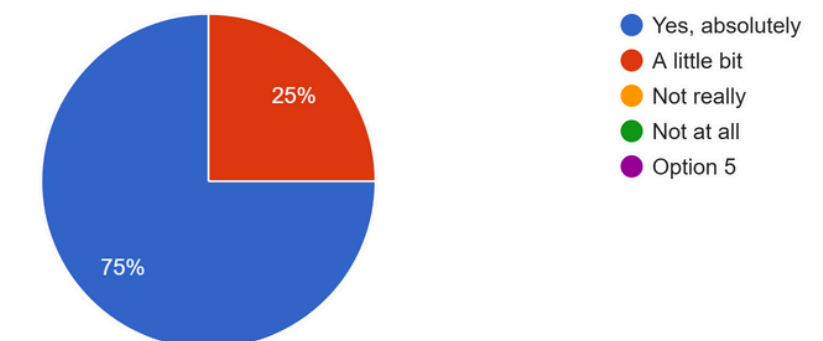
How engaging did you find the gamified elements (badges, XP, progress bars, levels, etc.)?

8 responses



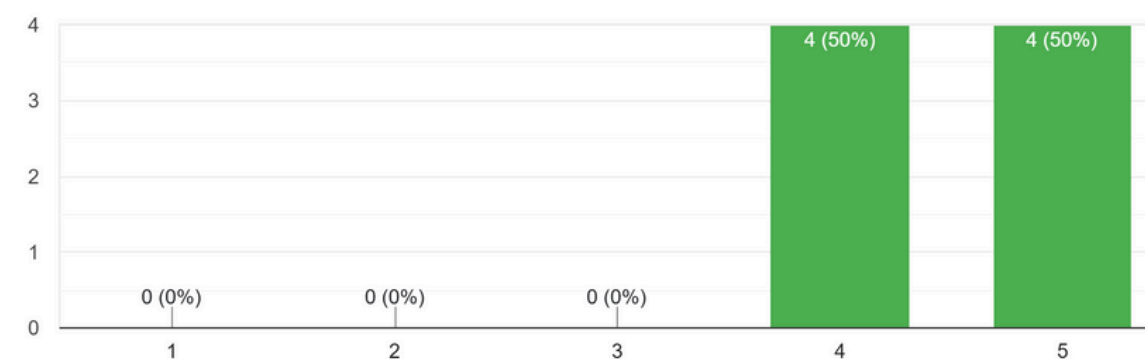
Do you feel that the gamification helped you stay motivated to use the platform?

8 responses



How would you rate your overall experience using the site?

8 responses

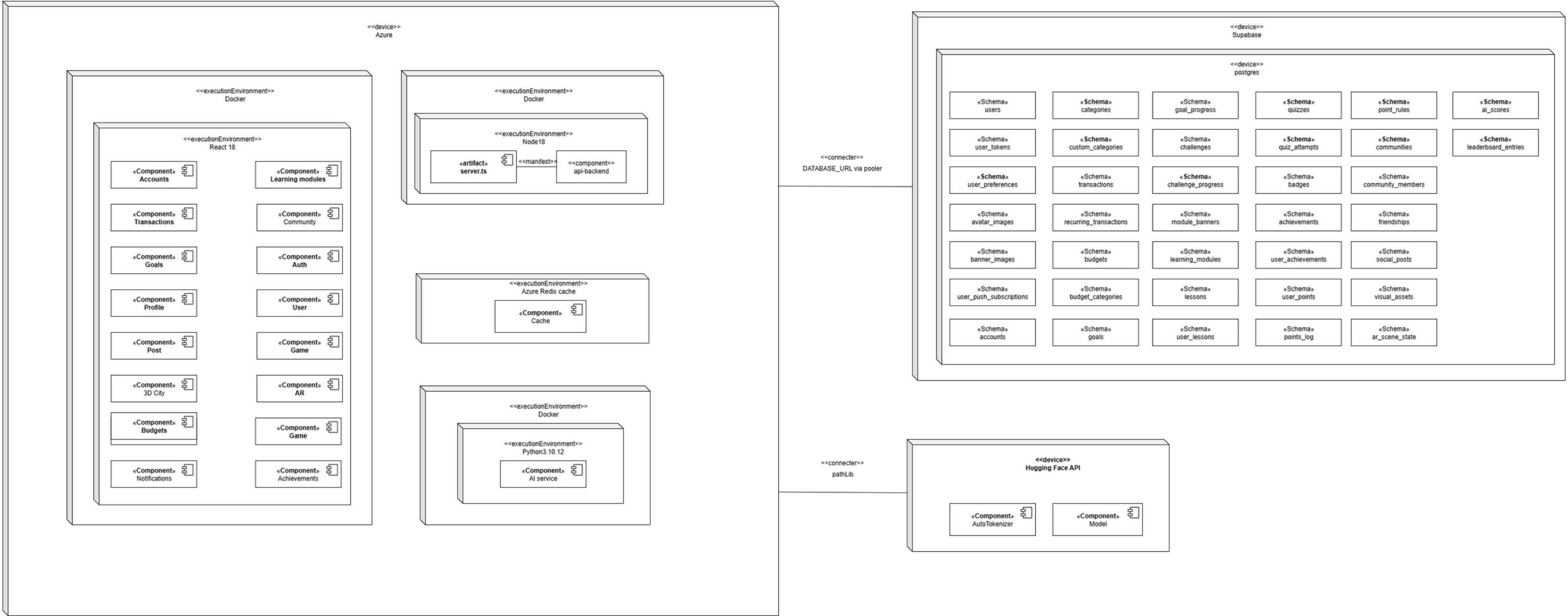


DEPLOYMENT



Deployment

DEPLOYMENT MODEL

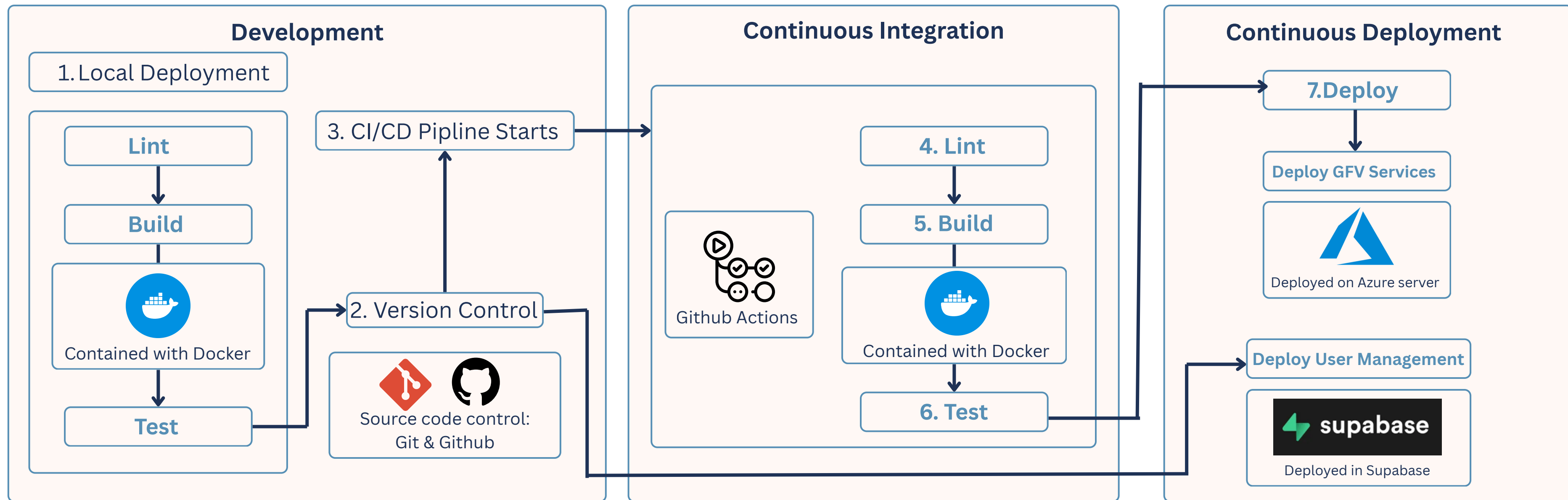


TESTING

CI/CD

A dark blue rounded rectangle with the text 'CI/CD' in white, surrounded by pixelated blue and white clouds. The clouds are stylized with a low-resolution, pixelated aesthetic. There are several cloud formations: a large one on the right, a medium one on the left, and a series of smaller ones along the bottom edge of the rectangle. The background is a light gray gradient with a dark blue header bar at the top containing the word 'TESTING' in white. There are also some light gray rounded shapes in the corners.

CI/CD PIPELINE



Q&A

Q&A



CODEBLOODED

Thank You.



EPI·USE[®]

Capstone 2025